

摘要：

围绕“模型变强后底层Harness是否会消失”的争论，前Kimi CLI负责人stdrc认为，Harness不会消亡，而是发生“复杂度向上迁移”：底层弥补模型缺陷的补丁式Harness逐步被模型内化（变薄），但面对复杂业务，解决多智能体协作、状态管理与上层管控的需求促使上层Harness变得更厚，最终作为协作基础设施隐形融入工作流。

最近 AI 编程圈挺有意思的，正在两极分化。

比如最近爆火的 Pi 走的是“极简Agent派”路线，Prompt 不到 1000 Token，就给 4 个基础工具，主张不折腾、省钱，让模型自己发挥；但另一边，大家又在做 Agent Swarm 和多智能体协作，系统越做越复杂。

这就带来一个圈内都在吵的问题：底层的 Harness 到底会不会消失？

很多人觉得 Pi 的爆火证明了“模型派”是对的，觉得大模型越来越强，外部的 Harness 迟早得消失。

但前 Kimi CLI 负责人、Raft 创始人 stdrc（Richard Qian）提了一个挺反常识的观点：**模型越强，Harness 反而越厚。**



stdrc
@istdrc



Show translation

显示翻译结果

我的 bet 是 agent harness 会越来越厚（或者说，针对 agent 的 experience design 和状态管理被延伸到了更深更广的方面），厚到有一天人们终于都理解了一切都是 agent harness，就像一切都是 human harness

Harness 这词还没火的时候，我就持有这个观点 [x.com/istdrc/status/...](https://x.com/istdrc/status/1804444444444444444)

事实上，我和 @xiaoxchan 几乎是在中国大模型公司中最早引入 harness 概念的人，Kimi CLI 从 0 到 1 也几乎没有参考过任何开源或闭源 agent 实现，system prompt、agent loop、toolset、产品形态都是从零生长的。而随后我们意识到 harness 需要延伸到 agent 间通信系统（也就是 Claude 近期推出的 session 间对话），进而有了 multi-agent harness 概念，也就是 Raft 的前身

只是这个“厚度”变了。以前 Harness 厚，是为了帮模型擦屁股、补能力，比如塞一堆 Prompt 和工具；以后 Harness 厚，是因为模型变强后要干复杂的脏活、累活，得靠它去解决多智能体之间的协作和上层管控。

简单来说，复杂度从“能力层”转移到“协作层”了。

被误解的 Harness

要看清 Harness 的演化逻辑，必须先破除行业内普遍存在的两层误解。

误解一：Harness 终将被训进模型，彻底消失



很多人有个误区，觉得大模型以后无敌了，Harness迟早会被训进模型里，彻底消失。

但只要你真正干过工程，就会发现这完全是倒果为因。stdrc 对此直接表示：“**说 harness 会被训到模型里的人，肯定是不做过 harness 的。**”

他从一线开发的角度拆了三点来解释这一问题：

首先，不是模型变强了淘汰 Harness，而是 Harness 先把路铺好了，模型才知道怎么学。就像人类社会，得先有了分工制度，大家才学会怎么协作。Harness 是模型的训练场，没有这个外部框架，模型根本没地方去练协作能力。

其次，智能越高，需要的工具反而越复杂。人类大脑进化了，没退化成原始人，反而搞出了法律和互联网。模型能力在变强，但它解锁的真实世界场景也越来越变态，底层能力被内化了，上层又会冒出更复杂的、需要 Harness 来管的新场景。

还有一点是单任务可以靠模型，复杂业务根本不现实。在面对长流程、多智能体、随时要看状态的真实业务时，纯靠大模型的脑子去记，必然会出现漏掉、搞错、甚至打架的情况。这不是模型不聪明的问题，而是系统工程问题，本来就不该让单个模型去硬扛。

❗ 误解二：模型越强，Harness 就会越薄

还有个流传很广的说法：模型越强，Harness就会越薄。

这个观点看起来挺对的，因为大家发现现在的底层提示词确实在变少，很多以前复杂的工具封装也没必要做了。

stdrc 认为这个观点“情有可原，但缺乏想象力”。它的问题在于，只看到了底层 Harness 的减法，没看到上层 Harness 的加法。



Show translation
显示翻译结果

说 harness 会被训到模型里的人肯定是不做过 harness 的，说 harness 会越来越薄的人情有可原，主要是缺点想象力

3:18 AM · Aug 10, 2026 · 592.8K Views

• 592.8 千次浏览量

不可否认，随着模型基础能力提升，大量“补丁式 Harness”会逐步退场：比如以前为了防止模型格式输出报错做的强制约束、为了教模型用工具写的长篇大论的 Prompt、还有各种死板的报错重试机制…… 这些为了弥补模型缺陷而存在的底层设计，确实会越来越薄，甚至完全消失。

但这并非 Harness 的消亡，而是复杂度的向上迁移。

当模型不需要人类程序员“擦屁股”时，它需要的，是要求你为它建立一套新的法则——更强的模型会解锁更复杂的任务场景，进而催生出新的Harness 需求：

比如怎么让好几个 Agent 互相打配合？跨会话的状态怎么同步？怎么做主动的记忆管理？动态权限和不同系统的对接标准怎么定？…… 这些需求在弱模型时代根本不会出现，自然也不会被



纳入大家对 Harness 的认知里。

重新理解 Harness：一场“阴阳共生”的动态演化

纠正这些误解后，我们需要回到核心问题：如何定义真正的 Harness？

大众常把 Harness 窄化为“System Prompt + 工具调用封装”。从完整的工程定义来看，**Harness 是包裹在模型外层的运行时工程管控层**，核心解决的是“模型如何稳定、持续、可控地与真实世界交互”的问题。它由四个核心要素构成：Agent 执行循环、上下文与状态管理、工具与资源调度、以及安全与边界治理。

而且 Harness 的范围一直在变大。从早期的模型弱，Harness 只管拼装简单提示词；到中期的模型变强，Harness 演化出多工具并行、子智能体调度；再到现在模型极强，Harness 开始搞智能体集群、跨系统状态同步、多 Agent 任务交接。

stdrc 将模型与 Harness 的关系比作“**阴与阳**”。这是一种此消彼长、共同进化的动态关系。

局部功能的“内化”，是阴的扩张。

当模型基础能力增强，一部分底层 Harness 会被模型取代。例如在 Kimi CLI 的实践中，stdrc 曾计划移除专用的 subagent 调度机制，改为由模型直接通过 bash 终端实现子任务拆分；同时移除原生的并行工具调用，改为由模型生成工具调用脚本来实现并行。

这是模型能力向外扩张的必然结果，也是外界感知到“Harness 变薄”的真实原因。

上层需求的“生长”，是阳的延伸。

模型能力每上一个台阶，就会解锁更复杂的业务场景，从而催生出更上层的 Harness 需求。单任务能力成熟后，催生了多智能体协作需求；单会话能力成熟后，催生了跨会话记忆与主动上下文回溯需求。在这场阴阳博弈中，智能水平越高，它与现实世界的摩擦力就越大。

这个过程没有终点。如同人类大脑比猿类更发达，于是，人类发明了语言、文字、计算机、互联网等更复杂的交互系统。智能水平越高，与世界的交互方式越复杂。Harness 就是 Agent 智能的“交互基础设施”，只会随智能升级而持续向外延伸。





译自中文翻译 显示原文

你可能不太理解“harness”到底是什么。

如果你指的是系统提示符和工具，那么模型当然可以学习它们，之后你只需要一个工具列表和相应的工具实现即可。更进一步，当模型功能更强大时，我们甚至只需要一个 bash 工具。我参与过 Kimi K2.5 模型的训练过程，并且从零开始编写过一个 Harness，最初只是一个空的系统提示符、工具集和一个循环——我当然明白人们所说的“将 Harness 训练到模型中”是什么意思。

但是，如果你回顾 Harness 的发展历史以及这个术语本身的出现，你会发现随着模型智能的提升，Harness 也在不断变得更加复杂：并行工具调用、子代理、代理群、代理团队、交接、主动压缩、通道、跨会话通信等等。你可能认为模型可以学习所有这些，但实际上，正是 Harness 的复杂性让模型能够学习它们。

随着模型的不断完善，所有这些功能确实都可以逐步淘汰——例如，我在开发 Kimi CLI 时，就准备移除子代理，用直接的 bash 调用（就像 pi 那样）来替代，并用工具调用脚本取代并行工具调用。但与此同时，更强大的智能也带来了更复杂的交互方式，例如引入主动式上下文回溯。

这是一个潮起潮落的过程，就像“阴阳”的比喻一样。x.com/nocommas/status/1811111111

如果你观察动物甚至人类的进化，你会发现道理是一样的：随着智力水平的提高，人类与世界互动的方式也变得越来越复杂。人类的大脑比猿类更聪明，一旦被自然选择保留下来，这并不意味着我们就停止使用木棍计数——而是催生了复杂的语言和文字。同样，当人类社会提升集体智慧水平时，人们并没有废除纸张；而是发明了计算机和互联网。

最终，人类将逐渐将自身与最初为“唯一智慧”而发展的社会基础设施融合起来。人类文明中的一切都可以被重塑（显然，我们已经看到所有 SaaS 产品都被重塑，而且很快我们还会看到更多——尽管这不仅仅局限于计算机软件）。

不要只关注模型训练的狭隘视角——不妨也看看人类学。

请为此翻译评分：

针对这场讨论，推特底下有一个开发者 Monk Zero 的评论十分有趣。

Monk Zero 从哲学层面，认为 Pi 方案的本质是“薄层提示词 + 厚层 Harness”。他指出模型如同狄俄尼索斯式的混沌与创造力，而 Harness 则是阿波罗式的秩序与结构。二者是永恒的互补。

I was able to comment. The truth about Pi's article actually just said thin prompt (your context) and thick harness

Harness is about capturing the invariant structural mechanism, the yang for yin(model), the Apollo for the Dionysus, the *Antigma* for the Enigma

我能够发表评论。关于 Pi 的文章的真相其实很简单：只是提到了“薄层提示”和“厚层约束”。

“驾驭”捕捉到那种不变的结构机制——即“阴”中的“阳”，以及“阿波罗”中的.....
狄俄尼索斯，即“谜团”的化身

无论是从stdrc的工程演化的视角，还是从Monk Zero的本质定义的视角，Harness都将必然存在，不会消失。

从 0 到 1 的验证：Kimi CLI 里的 Harness 生长史

stdrc的这套“复杂度迁移”的理论，并非纸上谈兵，而是基于 Kimi CLI 从 0 到 1 的完整工程实践。

早期的 Kimi CLI 没有参考任何开源框架，完全从零生长。从基础的单工具调用，慢慢长出并行调用、子智能体、状态管理。但

到了迭代后期，团队索性做了一次大胆的“减法实验”：把专门搞 subagent 调度的代码全砍了，原生的并行控制也不要了，全放手让模型自己用脚本去搞。

结果发现，只要模型能力到了那个临界点，底层的 Harness 确实可以丢弃。

而底层代码删完，团队就发现更头疼的问题。Harness 的边界不能只停留在单个 Agent 内部，多个智能体之间怎么通信？怎么交接任务？怎么搞定通信协议和分工？

于是，他们开始往上层加码，提出多智能体 Harness 的架构概念，搭建 Agent 任务交接、通信协议、分工机制，同时研究跨会话通信、主动上下文压缩、上下文回溯等状态管理技术。

这些探索，就是后来 Raft 的技术前身。而这个方向的判断，也与之后 Claude 推出跨会话对话、行业集体转向多智能体的趋势高度吻合。

正是这种从零生长的经历，让他们能清晰地观察到 Harness 复杂度的迁移规律，而不是被某一个静态阶段的认知困住。

创业践行：Raft 就是下一代“厚 Harness”

离开月之暗面后，stdrc 创办了 Raft。如果说 Kimi CLI 是一次演化实验，那 Raft 则是对“上层厚 Harness”架构的直接实践。

在做 Raft 过程中，stdrc 大胆去掉了自己曾经搭建的单 Agent 运行时。Raft 既不写 Agent 循环，也不封装基础工具，它直接把市面上成熟的 Claude Code、DeepSeek 等产品作为“团队成员”接入。

他的逻辑很简单，底层的单 Agent 能力已经足够成熟，模型已经能内化这部分 Harness，再重复造轮子没有价值。Harness 的下一个战场，不在单个 Agent 内部，而在多个 Agent 之间。

Raft 的全部火力集中在上层，如何解决多智能体协作的系统性难题。

具体来看，Raft 的“厚 Harness”体现在四个核心维度。

1.给 AI 发身份证和记忆力：模型只管单次回答，但 Raft 负责让每个 Agent 都有独立的进程、记忆和工作习惯。任务中途断了，换个时间还能无缝接上。

2.立规矩和分工：模型自己是不会主动搞团队协作的。Raft 把人类上班那一套搬了过来，用类似工作群的“频道”来隔离聊天，支持任务认领、认降和交接，所有步骤都留痕，方便人类随时查账。

3.打破厂商壁垒：Raft 搞了一套跨厂商的通信协议。让 Claude、DeepSeek 可以在同一个工作区里用统一的标准对话。这事单靠某一家模型公司是绝对不会去做的。

4.人类和 AI 一起上班的工作区：在这里，Harness 已经不是一段代码，而是变成了团队的工作流和权责体系。

连定价都挺有意思，每个 Agent 只算 0.1 个人类席位，支持跨团队共享。到这一步，Harness 已经从一个“技术工具”，变成了整个团队的日常沟通和协作规则本身。

最后，stdrc 对 Harness 的终局提了一个挺好玩的观点：“**当一切都是 Harness 的时候，它其实就隐形了。**”



比如在用 Raft 时，你不会觉得自己是在面对一套复杂的“Agent 调度后台”，它看起来就是一个把 AI 拉进来的办公软件。所有的状态同步、权限控制，全被藏在了产品直觉后面。

就像我们每天在公司上班，不会天天把“公司制度、法律是人类社会的 Harness”挂在嘴边一样。一旦 AI 员工真正普及了，这层最厚的 Harness 就会彻底融入日常工作流，变成像电网、自来水一样的基础设施。

所以回头看看最开始那个问题，“到底是模型吞噬一切还是 Harness 持续生长”，或许都低估了技术演进的维度。

真正的高手都在看高处，如何在更高维度的上层协作与复杂业务场景中，做出别人拿不走的新壁垒。



OneMoreSecond @MoreMoreSecond · Aug 10

单体模型终有上限，上限以外的无限天地都属于 harness

2

1

406



stdrc @istdrc · Aug 10

你懂了

373

来源：[AI科技评论](#)

风险提示及免责声明

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码
免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

40 收藏

分享到:  

写评论

请发表您的评论

 表情  图片

发表评论